Essai : Minimisation avec perte

```c
int contains_c_node(c_node *tree, c_node *node) {
  assert(tree != NULL && node != NULL);
  if (tree == node) return 1;
  if (is_leaf(tree)) return 0;
  for (int i = 0; i < MAX_CHILDREN; i++) {
    if (contains_c_node(tree->children[i], node)) return 1;
  }
  return 0;
}

/*
 * Finds the closest node to the given node in the given tree,
 * and returns the distance.
 * Note : the given node and tree cannot be leaves, the returned node also cannot be a leaf
 */
node_dist closest(c_node *tree, c_node *node, int ref_dist) {
  assert(node != tree);
  assert(tree != NULL && node != NULL);
  assert(!is_leaf(node) && !is_leaf(tree));

  if (has_only_leaves(tree)) {
    if(contains_c_node(tree, node)) {
      return (node_dist){NULL, INT_MAX};
    }
    return (node_dist){tree, c_mean_distance(tree, node)};
  }

  node_dist closests[MAX_CHILDREN];
  node_dist best = contains_c_node(tree, node) ? (node_dist){NULL, INT_MAX} : (node_dist){tr

  for (int i = 0; i < MAX_CHILDREN; i++) {
    if(best.distance <= ref_dist) break;
    if (tree->children[i] == node) continue;
    if (is_leaf(tree->children[i])) continue;
    closests[i] = closest(tree->children[i], node, ref_dist);
    if (best.node == NULL || closests[i].distance < best.distance) {
      best = closests[i];
    }
  }
  // printf("closest : %p\n", (void*)best.node);
  return best;
}
```

```
/*
 * Finds the closest pair of nodes in the given tree,
 * and returns the distance between them
 * Note : the tree can't be a leaf
 */
pair_dist closest_pair(c_node *tree, c_node *ref_tree, int ref_dist) {
  assert(!is_leaf(tree) && !is_leaf(ref_tree));
  pair_dist best = {NULL, NULL, -1};
  if(has_only_leaves(tree)) {
    node_dist n_d = closest(ref_tree, tree, ref_dist);
    pair_dist p_d = {tree, n_d.node, n_d.distance};
    best = p_d;
  } else {
    int i;
    for(i = 0; i < MAX_CHILDREN; i++) {
      if(best.distance == ref_dist) break;
      if(is_leaf(tree->children[i])) continue;
      node_dist n_d = closest(ref_tree, tree->children[i], ref_dist);
      pair_dist p_d1 = {tree->children[i], n_d.node, n_d.distance};
      pair_dist p_d2 = closest_pair(tree->children[i], ref_tree, ref_dist);
      best = p_d1.distance < p_d2.distance ? p_d1 : p_d2;
    }
  }
  // printf("closest pair : %p %p distance : %d\n", (void*)best.node1, (void*)best.node2, b
  return best;
}

c_node *replace_node(c_node *tree, c_node *old_node, c_node *new_node) {
  if (tree == old_node) {
    free_c_tree_and_cleanup(old_node);
    return new_node;
  }

  for (int i = 0; i < MAX_CHILDREN; i++) {
    if(is_leaf(tree->children[i])) continue;
    tree->children[i] = replace_node(tree->children[i], old_node, new_node);
  }
  return tree;
}

int step_minimize_c_tree_with_loss(c_node **tree, int ref_dist) {
  pair_dist closest_nodes = closest_pair(*tree, *tree, ref_dist);
  assert(closest_nodes.node1 != NULL && closest_nodes.node2 != NULL);
  printf("old node : %p\n", (void*)closest_nodes.node1);
  printf("new node : %p\n", (void*)closest_nodes.node2);
  *tree = replace_node(*tree, closest_nodes.node1, closest_nodes.node2);
```

```
  printf("replaced !\n");
  return closest_nodes.distance;
}
```